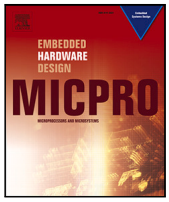
Microprocessors and Microsystems 109 (2024) 105084



Contents lists available at ScienceDirect

Microprocessors and Microsystems

journal homepage: www.elsevier.com/locate/micpro





# Count overflow and privilege mode filtering extension implementation on a RISC-V on-board processor

Andrea Fernández Gallego *, Miguel Jiménez Arribas, Iván Gamino del Río, Agustín Martínez Hellín, Manuel Prieto Mateo, Óscar Rodríguez Polo, Antonio da Silva, Pablo Parra, Sebastián Sánchez

*Space Research Group, Department of Automatics, University of Alcalá, Alcalá de Henares, 28805, Madrid, Spain*

ARTICLE INFO

*Keywords:*
CPU architecture
HPM
Performance measuring
PMU
RISC-V

ABSTRACT

*RISC-V* is a computer architecture that has recently attracted considerable attention due to its advantageous qualities: it is an open instruction set, based on reduced and simple instructions. For this reason it has become an appealing choice for a wide range of computing applications and has positioned it as a disruptive force in a wide variety of fields, including those that involve the development of safety–critical software, as in the space sector. The ability to evaluate the activities performed within a processor is of paramount importance in this type of systems to ensure the fulfillment of the requirements during space missions. The monitoring of these events inside the processor is managed by an instrument called Hardware Performance Monitor (HPM). This work shows the implementation of the *Sscofpmf* extension of the HPM compliant to the *RISC-V* privileged specification. The paper details the redesign of the existing performance counters from a *RISC-V* baseline version previously implemented. A comparison between the two versions of both resource utilization data and power consumption is also provided. As expected, the *Sscofpmf* extension version has a higher resource utilization. Nevertheless, the paper shows that the additional functionalities included in the system have been validated without any changes in the processor clock frequency, so the extension does not introduce any performance overhead.

## 1. Introduction

In recent years, an increasingly significant use of the *RISC-V* architecture is being observed in the context of both *European Space Agency* (ESA) missions [1] and industry-led space projects [2]. One of the key reasons for this adoption is that *RISC-V* provides not only a versatile architecture but also an open and free solution, so no license is required [3]. This fact enables a new era of processor innovation [4]. In addition, it allows the community to share technical investments, contribute to the strategic future, enjoy design freedom and substantially reduce the cost of innovation.

In line with this approach, the Space Research Group [5], in which this research has been carried out, has been working on its own *RISC-V* implementation [6] for the past few years. The SRG develops science and technologies for use in space applications [7]. Its principal areas of research are Solar Physics, development of flight electronics, development of flight software, test tools and control systems. Among others, within the *Solar Orbiter* [8] mission, the *Energetic Particle Detector* (EPD) instrument [9,10] is highlighted.

Specifically, the design of a *RISC-V* processor (RV32Xtrace) [6] was implemented by the SRG to improve the traceability of the executed software. This processor has been developed on a *Field Programmable Gate Array* (FPGA), which makes it possible to address the implementation of extensions such as the one presented in this article.

In those areas where safety–critical software needs to be developed, it is essential to have hardware to support the characterization of the software in different working scenarios. In order to satisfy this necessity, most modern processors have built-in hardware that monitors the performance of their events [11]. This data collection provides information about applications or the operating system. The monitoring events can be grouped into five categories: program characterization, memory accesses, pipeline stalls, pipeline branch prediction and resource utilization. This hardware is denominated *Hardware Performance Monitor* (HPM) or *Performance Monitoring Unit* (PMU).

Typically, the HPM design includes two components: performance event detectors and event counters. Each counter has its corresponding event associated. Events and counters usually have a mask that inhibits

* Corresponding author.
*E-mail address:* andrea.fernandezgall@uah.es (A.F. Gallego).

https://doi.org/10.1016/j.micpro.2024.105084
Received 21 December 2023; Received in revised form 15 April 2024; Accepted 8 July 2024
Available online 14 July 2024
0141-9331/© 2024 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

them. For instance, stop the counter of some events depending on the current processor privilege mode, or stop some events related to interruptions, so that these interruptions do not confluence with the test performed at some specific moment.

The HPM can also be used in the characterization of the program. This characterization defines the independent attributes of its implementation. For example, the number and type of instructions completed by the program. Another example are memory accesses which comprise most of the events. They can count, for example, references or misses when reaching the cache via the bus. Another relevant event, related to the content of this article, is the overflow status: an interruption is raised in case a counter overflows.

In the *RISC-V* privileged specification [12], *Control and Status Registers* (CSRs) hold configuration registers to manage or to indicate the status of different functionalities of the CPU. This privileged specification is part of the related specifications of the *RISC-V* Instruction Set Architecture. With regard to the HPM, it is implemented as these types of registers. Therefore, with the use of CSRs one can enable or disable specific HPM functionalities, configure its parameters and retrieve the performance data collected. The HPM accumulates these performance statistics in specific registers, and these values can be accessed and read from the CSRs by the CPU or software for analysis, profiling or debugging purposes. Within the RISC-V architecture, the interaction between the CSRs and the rest of the architecture is delimited by what is defined on the RISC-V specification. The specification defines the addresses from each register and that their access must be atomic. It also defines, for the HPM, two of the events which have to be monitored and some control registers. The rest of the events and how both the HPM and the CSRs are implemented in the microarchitecture is left platform dependent, which allows for some very interesting features to be developed. For example, synchronizing the counting of events with the execution of instructions, with the advantage that this entails during CPU and software development and testing [13].

Another important aspect from a *RISC-V* processor is that it can operate in one of several execution modes with various levels of privilege. These privilege levels dictate which instructions can be executed and which parts of memory can be accessed. The three major privilege levels for *RISC-V* are *user mode*, *supervisor mode*, and *machine mode*, in order of ascending privilege. Privilege levels provide protection between different components of the software stack, and attempts to perform operations that are not allowed at the current privilege level will cause an exception. These exceptions will typically cause traps in an underlying execution environment.

Machine mode has the highest privilege and is the only mandatory level for a *RISC-V* hardware platform. Code executed in machine mode is required to be very reliable, since it has low-level access to the machine implementation. This mode can be used to manage *RISC-V* safe execution environments. A program running in this mode can access all registers and memory locations. User mode and supervisor mode are designed for conventional application and operating system use, respectively.

Simpler *RISC-V* implementations may provide only machine mode, although this will not provide protection against incorrect or malicious application code. Many *RISC-V* implementations will also support at least user mode to protect the rest of the system from application code.

The work presented in this paper shows the implementation of the *Sscofpmf* extension of the HPM compliant to the *RISC-V* privileged specification. The paper details the redesign of the existing performance counters from a *RISC-V* baseline version previously implemented [13], comparing the two versions in terms of both resource utilization and power consumption. The extension has been performed on an FPGA over the *RISC-V* architecture-based demonstrator previously developed by the SRG group.

The implementation of the Sscofpmf extension presents interesting engineering possibilities. A prime example of its utility in the realm of space exploration is the introduction of analog functionality to those of watchdog timers which play a crucial role in ensuring system reliability and fault tolerance in spacecraft onboard computers. Traditionally, watchdog timers are software-controlled counters that reset periodically to prevent system failures. However, in cases where software execution is interrupted or fails to reset the timer, the watchdog triggers an interrupt to address the underlying issue. Likewise, with the Sscofpmf extension this functionality can be seamlessly integrated into the architecture, eliminating the need for external hardware. This closely aligns with the persistent pursuit in space mission development to minimize the external hardware components, thereby reducing power and mass demands. By assigning the desired duration of the subroutine directly within the architecture, the extension enables precise monitoring of critical processes and timely detection of anomalies. If the predetermined time threshold is exceeded, the extension generates an interrupt, facilitating prompt investigation and resolution of potential errors. Such techniques have already proven useful in recovering from errors encountered by the Mars rovers [14].

Additionally, by leveraging the programmable nature of the PMU counters, users can configure countdowns for diverse events without the reliance on software polling, thereby providing a mechanism for real-time event detection and response while offloading processing burdens to hardware components operating in parallel, and thus alleviating the computational load on the main system. Particularly noteworthy is the reduction in latency achieved through hardware-triggered interrupts. An analogous strategy, has already been previously demonstrated in existing literature, where the utilization of dedicated hardware for interrupt handling has shown promising results in reducing processor load and improving interrupt processing latency [15]. In scenarios such as obstacle detection during the traversal of planetary surfaces by exploration rovers, where event-driven computing is prevalent and timely response is critical, the ability to initiate interrupts based on concrete events ensures swifter and more efficient handling of mission-critical tasks [16]. This exemplifies the practical significance of the Sscofpmf extension in enhancing the performance and responsiveness of on-board computing systems in space exploration missions.

In the next section, the related works are introduced. A description of the new extension [17] of the privileged specification [12] along with the changes in the design based on the objectives of this work are presented in the third section. All the changes made for the implementation of the update in the *Performance Monitoring Unit* (PMU) (same as HPM) are exposed, with their corresponding tests, ending with the extension of the handler for exceptions and interrupts. In the results section, the functionalities of the original CPU (*Central Processing Unit*) with the new one described afterwards are compared. Finally, the conclusions are drawn.

## 2. Related works

As mentioned above, *RISC-V* has gained significance on the space field. However there are some present and future needs that need to be addressed, for instance, to meet the different requirements in applications in satellite data systems. S. D. Mascio et al. [18] present a roadmap to address those needs. For these requirements, four different types of processors are identified: low area/low power microcontrollers, on-board computers, general purpose processors for payloads, and enhanced payload processors for artificial intelligence. Several *RISC-V* based solutions are proposed for each of these processor types and compared to proprietary, space-grade commercial solutions. An extensive analysis of available results in the literature is carried out to demonstrate that *RISC-V* has the potential to solve a wide range of needs. This paper also shows the unprecedented number of open-source implementations and models that were developed in a brief time on a single instruction set architecture. Future space systems could benefit from many of these developments, and S. D. Mascio et al. identify and highlight what is still missing to meet the specific

needs of processors for space, especially in terms of fault tolerance and technology readiness level.

Moving to the main topic in the present article, it is known that most modern high-performance processors have on-chip space hardware that monitors processor performance. The data collected by this hardware provides performance information about the applications, the operating system, and the processor. This data can guide performance improvement efforts by providing information that helps programmers tune the algorithms used by the applications, the operating system, and the code sequences that implement those algorithms. The basic concepts of a PMU are presented in [11].

Execution data is valuable for identifying performance bottlenecks (the characteristic that most affects performance) and guiding optimizations. Periodic sampling of a processor's performance monitoring hardware is an efficient and unobtrusive way to obtain detailed profiles. Unfortunately, existing hardware simply counts events, such as cache misses and fork mispredictions, and cannot accurately attribute these events to instructions. An alternative approach, called ProfileMe, which samples instructions, is proposed in paper [19]. As a sampled instruction moves through the processor pipeline, a detailed log of all interesting events and latencies of the pipeline stage is collected. ProfileMe also supports paired sampling, which captures information about interactions between concurrent instructions, revealing useful concurrency information and the utilization of multiple pipeline stages while an instruction is in flight. An inexpensive hardware implementation of ProfileMe is defined, a variety of software techniques for extracting useful profile information from hardware, and explain several ways in which this information can provide valuable feedback to programmers and optimizers.

Due to the increasing complexity of software in embedded systems, performance aspects are becoming increasingly important. Often, execution times and events are not easily countable or measurable due to the lack of functionality in these systems. Runtime monitoring is also relevant in terms of reacting to internal and external events dynamically. This is an important discipline especially for systems that use multiple tasks with internal or external resource dependencies. Another problem is that measurements during the development process are often performed by interfering with the system. This method leads to biases in the measurement results, since the final system is implemented without these interfering functionalities and, therefore, may perform more efficiently than the system used during development. The document elaborated by T. Scheipel et al. [20] shows a system-aware PMU. The purpose is to develop a module in a hardware definition language capable of measuring execution times and task-aware and unaware events, parallel to the nominal execution of the system. As it is said in [20]: "The measurements of this module should be delivered to the programmer through an easily accessible interface. The main focuses of this project are scalability, platform independence with respect to processor and operating system (OS), as well as easy extensibility". They also provide information about reusability of the counters during runtime.

## 3. Development

In this section, first, a description of the new extension [17] of the privileged specification [12] is given. Subsequently, the modifications to the design of the existing *Performance Monitoring Unit* (PMU) [13] are exposed based on the context of the implementation of the updated functionality. Finally, their corresponding tests are discussed, ending with the extension of the handling of exceptions and interrupts.

It is important to highlight that the implementation described in this article is fully compatible with other CPUs hosting the standard PMU described in the RISC-V specifications. Therefore, any processor utilizing the RISC-V architecture that integrates a PMU could potentially benefit from the modifications outlined here.

### 3.1. PMU update

The machine mode includes a *Hardware Performance Monitor* (HPM) or statistical unit. The HPM consists of several registers, such as the CSR called *mcycle*, which counts the number of cycles executed by the processor core. Another relevant register is the CSR called *minstret*, that counts the number of instructions retired by the core. These two registers are 64-bit accurate (32 high side and 32 low side) in all the RV32 (*RISC-V* 32 bit architecture) systems. Each register corresponds to a certain event, in turn, each event has an associated counter.

The rest of the HPM includes 29 event counters, *mhpmcounter3* - *mhpmcounter31*, each of 64 bits (split into high and low parts). The CSRs that are responsible for indicating the corresponding event, *mhpmevent3* - *mhpmevent31*, each with only 32 bits of extension. They are *Write Any-Read Legal* (WARL) registers that control which event has caused the corresponding counter increment. In other words, each *mhpmeventn* is related to each *mhpmcountern*.

The CSRs of the *mhpmevent* vector, as specified in *Volume II: Privileged Architecture* [12], enable the selection and control of the event count for their associated *hpmcounter* registers, providing flexibility in count tuning. All counters and configuration registers can be written with any desired value using the instructions outlined in the specification. Notably, these writes to the CSRs become effective upon completion of the respective instruction.

The changes implemented on the HPM in this article are based on an extension of the privileged specification mentioned above. This extension is called *RISC-V Count Overflow and Mode-Based Filtering extension* [17]. It addresses two basic needs that were in high demand by the *RISC-V* user community. These shortcomings refer to a new functionality, due to the possibility of the counters to overflow. This new functionality detects overflow in the HPM counters, and also filters and masks both the counting of events and interruptions caused by this overflow. Relevant changes fix the absence of the high part of the *mhpmevent* vector and the absence of a bit to serve as a request for the overflow interrupt in both *mip* and *mie*.

This overflow interruption would mainly be used as a countdown timer. For this purpose, the counter is set to 0 minus the desired value, for instance, $0 - 5 = -5$. This translated to the value of the counter would be 0xFFFFFFFFFB. Therefore, when the event associated with that counter is activated five times, an interrupt would be triggered, notifying the overflow of that counter. This functionality did not exist before in hardware, having to be monitored by software, and therefore intrusive to the execution of the rest of the program.

The name of the extension is "Sscofpmf" ('Ss' for supervisor level and privileged arc extensions, and 'cofpmf' for *Count OverFlow* and *Privilege Mode Filtering*). The new counter's count overflow interrupt will be treated as a standard local interrupt mapped to bit 13 in the *mip/mie* registers.

This extension expands the HPM description and extends the *mhpmevent* registers to 64 bits (in RV32). The *mhpmeventn* registers are WARL, they control which event causes its corresponding counter to increment and what happens when the counter overflows. Currently only a few bits are defined. The lower half bits of *mhpmevent* will be used for event selection.

The CSRs *mcycle*, *minstret*, *mhpmcountern*, and *mhpmeventn* access the first 32 bits, while the CSRs *mcycleh*, *minstreth*, *mhpmcounternh*, and *mhpmeventnh* access the high part (63-32 bits) of the corresponding counter or event. As mentioned above, the specification of the new extension [17] expanded the *mhpmevent* registers from 32 to 64 bits, and thus the *mhpmeventnh* registers must be added to the platform. For this purpose, when adding these registers, addresses $0 \times 723$ to $0 \times 73F$ have been assigned to them. On the other hand, the declared bits that make up the high part of these registers are shown in Table 1. To ease comprehension of the changes specified by the new extension, a scheme of the modifications implemented is shown in Fig. 3.

**Table 1**
Declared bits in *mhpmeventh* registers [17].

| Bit | ID | Description |
|---|---|---|
| 63 | OF | Overflow status and interrupt disable bit that is set when counter overflows. |
| 62 | MINH | If set, then counting of events in M-mode is inhibited. |
| 61 | SINH | If set, then counting of events in S/HS-mode is inhibited. |
| 60 | UINH | If set, then counting of events in U-mode is inhibited. |
| 59 | VSINH | If set, then counting of events in VS-mode is inhibited. |
| 58 | VUINH | If set, then counting of events in VU-mode is inhibited. |
| 57 | 0 | Reserved for possible future privilege modes. |
| 56 | 0 | Reserved for possible future privilege modes. |

The OF bit is set when the corresponding *hpmcounter* counter overflows, and is held until written by software. Since the *hpmcounter* values are unsigned, the overflow of the counter bits is defined unsigned. There is no loss of information after an overflow, since the counter resets and continues counting as long as the OF bit remains set.

If any *hpmcounter* overflows while the associated OF bit value is zero, then a counter overflow interrupt request is generated. If the OF bit value is one, no request is generated. Consequently, the OF bit works as a switch that disables the interrupt generated by the associated *hpmcounter*, as it is described in [17]. However, during our implementation it has been found that during the handling process this presents a problem, as it is not possible to differentiate between those OF bits that have been set to disable interrupts and those that have been overflowed. This problem is something that the specification actually predicts, but the provision is established as non-mandatory. Their solution is that, as the configuration of interrupt disabling is done by software, this software should have in a stored structure which counters are disabled and which can receive an overflow interrupt, in order to differentiate them during processing. We propose a bitmask structure where each bit corresponds to one of the counters, enabling the bit of that corresponding counter that can receive overflow interrupts, but it could be any other valid structure.

Another problem found was that counters were not filtered by privilege mode, and thus they continued to increment while traps to more privileged code were handled. This causes two problems: It introduces unpredictable noise to the counter values observed by the user, and it leaks information about privileged software execution to user mode. The extension remedies these issues by introducing privilege mode filtering for the counters. The MINH and UINH bits are capable of enabling or disabling the counting of events in machine and user mode respectively.

VSINH and VUINH stand for virtual supervisor and user inhibit bits respectively. As VS and VU modes are not implemented in the platform, these bits are declared but hardwired to zero and therefore not used.

Writing to the *mhpmcountern* or *mhpmeventn* registers never causes a counter overflow. What causes this overflow is the hardware incrementing of the counter registers.

This interrupt request signal is treated as a local interrupt, the LCOFIP bits of the *mip* and LCOFIE of the *mie* are respectively the enable and pending bits for this interrupt ('LCOFI' stands for '*Local Count Overflow Interrupt*').

The generation of an interrupt request by an *hpmcounter* sets the LCOFIP bit in the *mip* registers and sets the associated OF bit. The LCOFIP bit is cleared by hardware and software during the trap handler, when one or more counters overflow.

Finally, regarding the priority of the new type of interrupts, they have the lowest priority and, moreover, follow a decreasing order according to their privilege level: machine mode interrupts are handled first, then supervisor mode interrupts and, lastly, user mode interrupts. When several interrupts of the same privilege are launched simultaneously, they shall obey the following order, as specified in [17]: Multiple simultaneous interrupts destined for the same privilege mode are handled in the following order of decreasing priority: MEI (Machine External Interrupt), MSI (Machine Software Interrupt), MTI (Machine Timer Interrupt), LCOFI (Local Count Overflow Interrupt). Additionally, if more than one overflow interrupt were to occur at the same time, this would cause the same situation described earlier, with the management software needing to decide which interrupt to handle first.

### 3.2. Changes implemented

To be compliant with the updated specification, our implementation underwent several modifications, which will now be discussed. Fig. 1 illustrates the structural diagram of the interconnection of the relevant components, while Fig. 3 provides a visual representation of the specific modifications made.

One of the changes that has been added is the LCOFIP bit to the *mip* CSR as bit 13, to indicate the pending status of the new interrupt. In addition, the LCOFIE bit is added to the *mie* CSR, also as bit 13. The LCOFIE bit can be used to enable or disable the jumping of these interrupts. With these general configuration registers, all overflows are enabled or disabled at the same time, there is no specific bit for each counter. Instead, this can be done with the OF field of each *mhpmeventnh* CSR, as explained in Table 1.

On our design, the system constants are declared on a separated file. In this way, in case of a possible change in the architecture, this modification becomes lighter. The following changes are made:

- The high part of the *mhpmeventnh* vector is added, with their addresses ranging from $0 \times 723$ to $0 \times 73F$.
- The arrays *mhpmevent_t* and *mhpmcountersh_t* are created with the dimensions corresponding to the number of counters, per XLEN, i.e., at the moment, $13 \times 32$.
- To control the overflow of these last arrays, for each counter, it is necessary to know whether bit 32 is set to 1 or not. Its length is then modified from 32 bits to 33.

The functionality of the PLIC (*Platform Local Interrupt Controller*) module consists in the detection of all supported interrupts, whereas the CLINT is in charge of core exceptions (*Core Local INTerrupt*). With the help of the PLIC-CLINT multiplexer and the *PipeLine Logic module* (PPL), the processing of non-nominal processor flow (interrupts and exceptions) is carried out. The signal corresponding to LCOFIE is added to this unit. If this signal is set to 1, the corresponding interrupt processing of the interruption is carried out.

The CSRs unit is responsible for managing the processor status control registers. It performs three different functions: It enables atomic read/write access to the CSRs, it increments the PMU counters, and it supports the operation of CLINT exceptions. Fig. 1 illustrates the interconnections among the processor components and its relation with the explained units.

According to the changes marked in the specification, the OF signal is added and initialized to 0. Each event will have an associated OF signal, and instead of passing this information to the PLIC, the *of_general* signal is created (Fig. 2), which is one bit long. This signal corresponds to the logical addition of all the OF bits, i.e. it will take the value of 1 if at least one value of the OF signal is at 1, that is, if any of the OF bits has gone from 0 to 1 (see "Filter" box in Fig. 2). In this way, with a single bit, which will be passed to the PLIC module, the interrupt handling can be done.

When incrementing the counters, if any of them overflows, the corresponding bit of the OF signal counter sets to 1. Finally, one

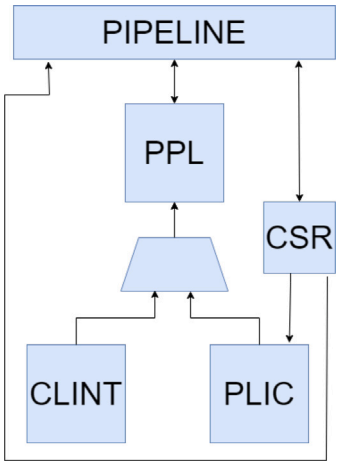



**Fig. 1.** Some pipeline stages require additional logic to their input and output signals. This logic comes from the Pipeline Logic module (PPL). Additionally, the information regarding non-nominal flow comes from a multiplexor that chooses between the CLINT and PLIC modules. Lastly, to determine the status of the processor, the CSRs are connected to the some parts of the pipeline and to the PLIC module. Arrows show data flow directions.

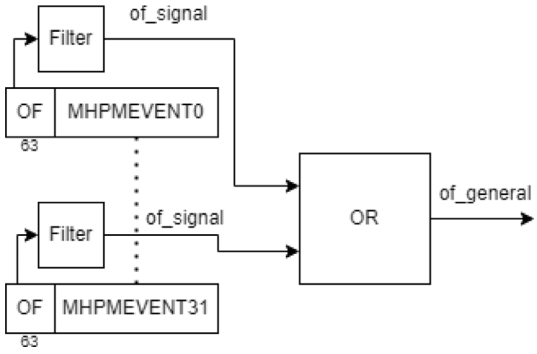



**Fig. 2.** Creation of_general. Arrows show data flow directions.

thing to note is that, according to the specification we have implemented [12], this process is only performed for the counters associated with *mhpmeventn* registers, ie.: only for *mhpmcounter3-31*. We argue that, likewise, it would also be useful to have this feature implemented for the *mcycle* and *mintstret* counters and as such, the implementation presented includes this functionality, as will be seen during verification. This way, interruptions will trigger whenever the any of the counters overflow. *RISC-V* is aware of this issue, and a new update of the HPM was recently developed to correct it [21].

Therefore, to support this new functionality within the CSRs unit, it is necessary to perform some additional checks. These modifications are detailed in Fig. 3. Thus, to increment any of the counters, the following verifications are performed. In the first place, it is checked if the count inhibition is disabled (by means of the *mcountinhibit* CSR), in that case, the next condition is verified. As mentioned above, if the MINH and UINH indexes are set to one, it means that counting is inhibited in machine mode and user mode, respectively. Therefore, in case the processor is currently in machine mode, and if the count is not disabled in this mode; or in case of being in user mode, and if the count is not disabled in that mode, it would be possible to continue with the process of updating the counters. If the event is active, the increment in the lower part of the counter, the first 32 bits, is performed. Subsequently, it is checked if the low counter has overflowed, that means bit 32 is set to one, in which case a bit is added to the high part. The bit 32 of the low part is then set to zero.

Now the detection of overflow at the top of the counter comes into play. As mentioned above, in case bit 32 of the high part of the counter is set to one, i.e. this counter has overflowed, the OF signal is set to zero. Subsequently, the *of_variable* signal is set to one, if the OF bit is set to zero, which will be used to activate the handling of this interrupt. Consequently, the OF bit is then set to one. This is done because what is of interest is the change in the OF signal, the change from zero to one. Finally, bit 32 of the high part of the counter is set back to zero.

To end with the changes, the reading and writing of these interrupts is enabled. In addition, the corresponding bit of the signal is set to zero.

The *IFace* file contains all the input/output interfaces of each component. Therefore, the signal called *of_general* is added there. This will be used to check in the PLIC whether it is set to one or not. If it is set to one, no request is generated.

### 3.3. Verification tests

Four tests were performed in order to verify that the overflow interrupts were triggered correctly, and that counts in machine or user mode can be disabled by means of the MINH and UINH signals.

First of all, a test was performed to check the overflow of the counter associated with the eighth event, corresponding to the *mhpmcounter7*, (active when an instruction of conditional jump is not taken). The address to be jumped to in the event of an interruption or exception can be configured. This value is stored in the *mtvec* register, as it will be explained later. Subsequently, both the high part of the counter, *mhpmcounter7h*, and the low part, *mhpmcounter7*, are loaded to their maximum value. Then, the instruction "BNE x0 x0 32" is performed, which produces a jump in case registers R0 and R0 are not equal, which obviously cannot happen. Therefore, as the branch is not taken, the counter increments in 1, and hence overflows. When the interruption occurs, it jumps to the address marked by *mtvec* and the handling begins. This process consists of setting the bit corresponding to this event back to zero (bit 32 of the counter) and resuming nominal execution using the *mret* instruction, returning to nominal execution.

A second test was performed, similar to the previous one but trying to test the worst possible corner case, to verify the robustness of the implementation. In this test, the counter associated with the eighth event, *mcycle* and *minstret* overflow at the same time. As in the previous test, the counters are loaded to their corresponding values, causing all three counters to overflow at the same time. The results were as expected, obtaining that when all three interruptions occur at the same time, they are handled one at a time (the handling is passed through three times).

Finally, two tests were performed, one in machine mode and the other in user mode to test the MINH and UINH signals. The first part of the machine mode test is the same as the previous test. When returning from the handling process, each bit 30, corresponding to the MINH signal of CSR_MHPMEVENT0H (*mcycle*), CSR_MHPMEVENT2H (*minstret*) and the eighth event, CSR_MHPMEVENT7H, are configured to 1. Therefore, after the handling process, it is verified that indeed these counters do not increment in machine mode, which is the mode in which the whole test is performed.

The fourth test is analogous to the previous one, but instead of machine mode, testing the behavior in user mode. However, a problem arises, because if an attempt is made to access registers while in user mode, an exception is generated and the system switches to machine mode, as they are only accessible in a trusted environment. To avoid this, an exception is generated on purpose, and from this machine mode, the necessary registers are modified. Each bit 28, corresponding to the UINH signal of the CSRs CSR_MHPMEVENT0H (*mcycle*), CSR_MHPMEVENT2H (*minstret*) and CSR_MHPMEVENT7H corresponding to the eighth event, are set to 1. After processing this exception, the privilege mode is switched to user mode, and it is verified indeed that these counters stop counting in this privilege mode.

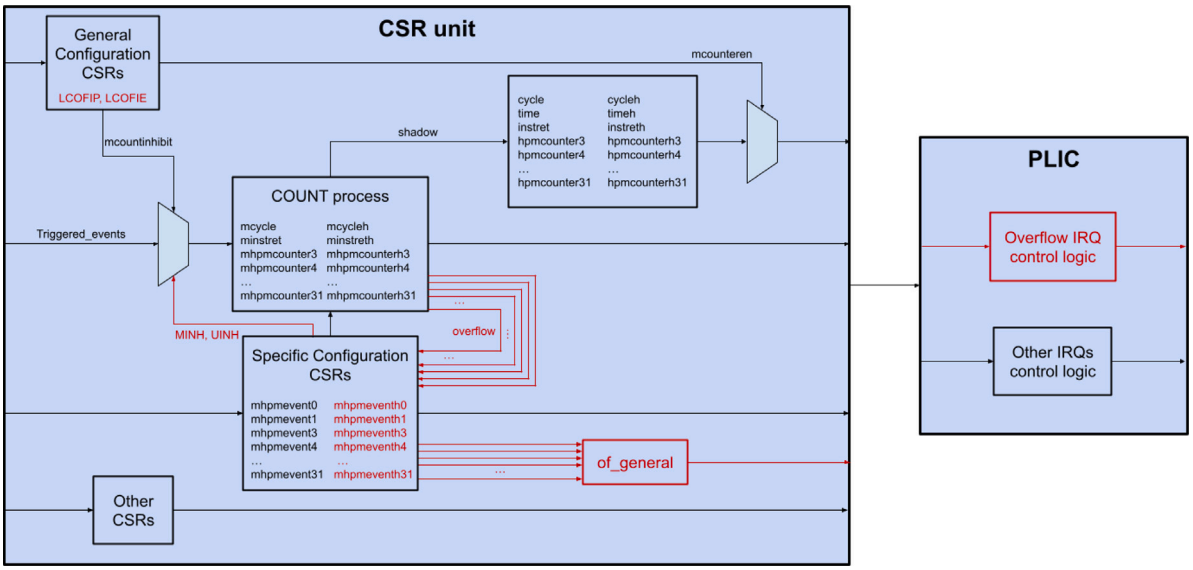



**Fig. 3.** Schematic representation illustrating the modifications proposed in this article, highlighted in red. As elucidated, the enhancements encompass the incorporation of additional configuration registers within the CSR unit. These registers facilitate the creation of a filter mechanism designed to suppress counter increments based on privilege levels, as well as the capability to trigger interrupts upon counter overflow. Furthermore, novel logic has been implemented to manage the handling of these newly introduced interrupts within the PLIC module. Arrows show data flow directions. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

**Table 2**
Currently supported *traps* on the platform.

| Interrupt | Exception code | Description |
|---|---|---|
| 1 | 7 | Machine timer interrupt |
| 1 | 11 | Machine external interrupt |
| 1 | 13 | Local counter overflow interrupt |
| 0 | 2 | Illegal instruction |
| 0 | 8 | Environment call from U-mode |
| 0 | 11 | Environment call from M-mode |

### 3.4. Trap handler

Trap controllers mainly use three control and status registers (CSRs) to manage traps: *mtvec*, *mcause*, and *mepc*. The *mtvec* register (machine trap-vector base-address) contains the address of the code in charge of handling the exception. When an exception occurs, the processor saves the exception code in *mcause*, saves the PC of this instruction in the *mepc* register (machine exception PC), and jumps to the address preconfigured in *mtvec* to perform the handling. Bit 31 of *mcause* indicates whether it is an interrupt (1) or an exception (0). Table 2 shows the *traps* that are currently supported on the platform.

When the processor detects a trap, in the first place, it jumps to the address where the exception treatment in *mtvec* begins. Then, the register is saved and the *csrr* instruction is used to find the cause of the exception (encoded in *mcause*) and respond appropriately. When the handling is finished, it restores the memory registers or returns to the nominal execution of the program using *mret*, which jumps to the *mepc* address. In case of an exception, *mepc* value is previously incremented by 4. If this were not done, the program will return to the instruction which generated the exception, and then a new exception will be raised, becoming a loop. In case of an interruption, the *mepc* is not incremented for the same reason. The instruction in the *mepc* address did not cause the interruption, and was not executed.

A refactoring of the current *traps* treatment code in the platform has been carried out, abstracting it to C code, obtaining a more manageable and complete code. It should be noted that the existing code was made in assembly language. This new code joins the existing traps and is composed of three main functions. When a *trap* is detected, they are first disabled, so that the handling process is not interrupted by a new interruption or exception. Subsequently, the handling is performed and, finally, they are re-enabled.

To disable the traps, the function that disables them creates a variable called *mstatus* (Machine-mode status), with the value of the CSR_MSTATUS stored in it. This CSR tracks and controls the current operational status of the core. The third bit corresponds to MIE (Machine interrupt enable). If this bit is set to 1, machine-mode traps are enabled.

The purpose of disabling interrupts and exceptions is to be sure that no other *trap* is raised during the handling process. For this purpose, the third bit, corresponding to the MIE, is set to 0. To do this, an AND operation is performed between the *mstatus* vector and a vector of the same length (32 bits), which has all its bits set to 1, except the third bit which is set to zero. This clears only the third bit, keeping the rest of the bits with their original value. Finally, this new vector is stored in the corresponding CSR. After the handling process, the interrupts are re-enabled. In this case, bit 3 corresponding to the MIE is set to 1, for this an OR operation is performed between the vector *mstatus* and a vector of length 32 bits, with all its bits at 0 except the third one which is a 1.

After disabling the traps, they are processed, as shown in Fig. 4. A distinction is made between the three interrupts and the three exceptions shown in Table 2 for handling, plus an extra case for the traps not covered. For exception handling, first check that bit 31 of *mcause* is set to 0, which indicates that it is an exception. Subsequently, a distinction is made between machine or user mode system call and illegal instruction. In these cases, the cause of the exception is printed together with the *mcause* value. Finally, a distinction is made between exceptions that are not covered. In this case an error text is printed, together with the values of *mcause*, *mepc* and *mtval*. For exceptions, when returning to the nominal execution, a value of 4 is added to the *mepc* in order to return to the next instruction from the one that caused the exception.

For interrupt handling, first, bit 31 of the *mcause* must be 1, which indicates that the *trap* is an interrupt. Subsequently, a distinction is made between a timer interrupt, an external interrupt or an interrupt implemented in the case of counter overflow. In the timer interrupt, the timer can be programmed to trigger an interrupt periodically when needed. In this case, it is reprogrammed to restart 60 s after the interruption. In the case of an external interrupt, the cause of the interrupt

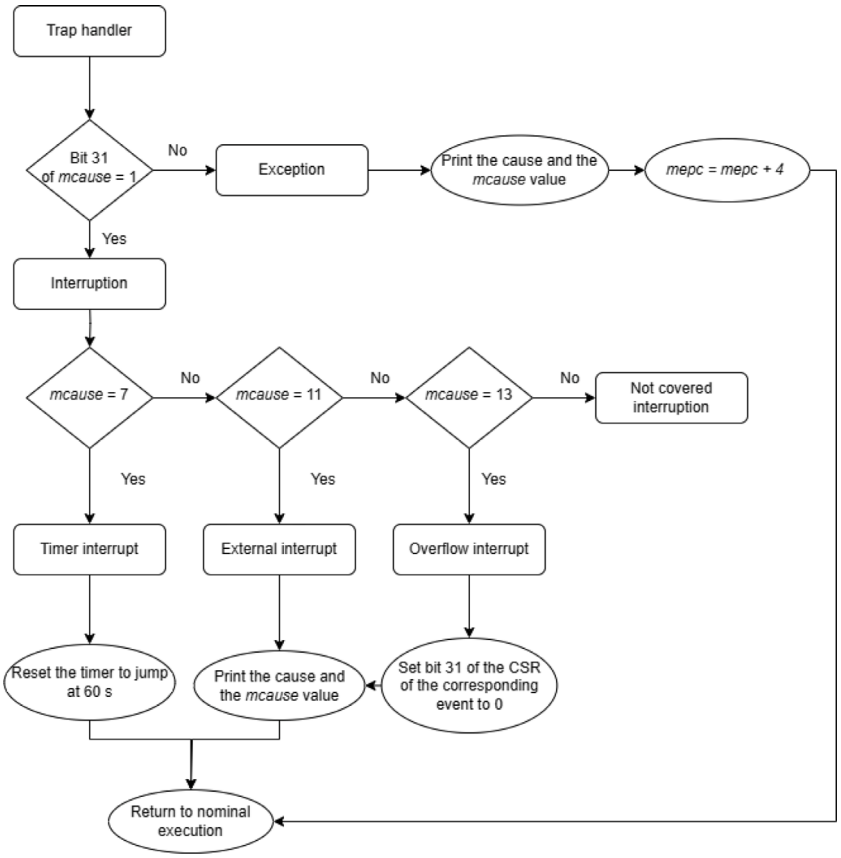



**Fig. 4.** Flowchart of the handling process. Arrows show data flow directions.

is printed together with its *mcause* value. Finally, for interruptions due to overflow, the cause is also printed, and in addition, it is identified which of the 32 counters has caused the overflow, and bit 31 of this CSR is set to 0 by means of an AND operation. As mentioned before, it is not trivial to identify the counter which triggered the overflow, since it is not possible to differentiate between those OF bits that have been set to disable interrupts and those that have been overflowed. For interrupts, nominal execution continues in the instruction that was being executed at the time the interrupt was triggered.

## 4. Results

This section compares the resources of the original CPU (without the changes of the new specification) with the new one. The resources used both in synthesis and implementation will be compared, as well as the actual power consumption, verified on a board, in particular the NEXYS A7. The expected result in this case would be that the CPU with the new implementation changes uses more resources than the original CPU, due to the increase in monitoring logic, such as an increase in the number of CSRs.

The resource utilization data of each CPU is obtained in both synthesis and implementation using the Vivado tool called *report utilization*. The results are expected to show higher utilization in synthesis than those used in implementation. This may not always be the case, but due to the specific circumstances of our project and the size of the board where we are implementing it, since there are resources to spare when implementing, Vivado does not need to use any resource-intensive techniques, thus always giving lower measurements. Synthesis generates a netlist that contains information about the general logic blocks being used and how they are connected. The implementation maps those generic components to physical ones found in the FPGA device (placing) and establishes the connections between them as defined in the netlist using physical routing resources (routing). In more detail, synthesis takes the circuit design and optimizes it. This “converts” the generic components into the primitives available on the FPGA, most notably LUTs and FFs. The implemented design again translates the design, this time from using individual primitives to using entire slices and CLBs, as they are available on the FPGA. These configured CLBs then have their locations set on the FPGA (placement), and connections are drawn between them (routing).

Despite the existence of a tool in Vivado meant for estimating the power consumption used, this has been obtained by making real measurements with a power supply because it is a more reliable measure. Measurements will be taken of the base power consumption of the board, of the base power consumption with USB connection to the hardware server within Vivado and, for each CPU version. Measurements will be taken while running different programs. The data will be analyzed looking for the increase of resources and the consumption variations.

As mentioned above, comparing the utilization data of the old PMU, it appears that the resources used are higher in the case of synthesis. For example, 5850 Slice LUTs (Table 3) are used in implementation, compared to 5976 (Table 4) in synthesis.

In the case of the new PMU, the result is the same: more resources are used on synthesis than on implementation.

However, for this work, the comparison between the old and the new PMU is essential. For both the synthesis and implementation cases, a higher resource utilization is observed for the new PMU, an increase of 39.7% in synthesis (Tables 4 and 6) and 32.4% in implementation (Tables 3 and 5). In particular, this is mostly due to the increased use of CSRs. These CSRs were implemented to expand, as explained above, the functionality of the HPM. Changes can also be seen in the increase in logic. This is due to the increase, for instance, of combinational blocks (ifs). These have been used for monitoring the overflow state, or for

**Table 3**
Implementation, old PMU.

| Component | Slice LUTs | F7 Muxes | F8 Muxes | Block RAM |
|---|---|---|---|---|
| RV32 | 5850 | 271 | 1 | 2 |
| clint | 73 | 0 | 0 | 0 |
| decode | 299 | 0 | 0 | 0 |
| execute | 634 | 1 | 0 | 0 |
| if | 21 | 0 | 0 | 0 |
| mc | 165 | 0 | 0 | 0 |
| memory | 96 | 0 | 0 | 0 |
| pipeline_logic | 379 | 0 | 0 | 0 |
| plic | 21 | 0 | 0 | 0 |
| prom | 0 | 0 | 0 | 2 |
| reg | 3556 | 270 | 1 | 0 |
| CSR | 2631 | 14 | 1 | 0 |
| GPR | 925 | 256 | 0 | 0 |
| rexmem | 58 | 0 | 0 | 0 |
| ridex | 117 | 0 | 0 | 0 |

**Table 4**
Synthesis, old PMU.

| Component | Slice LUTs | F7 Muxes | F8 Muxes | Block RAM |
|---|---|---|---|---|
| RV32 | 5976 | 287 | 1 | 2 |
| clint | 25 | 0 | 0 | 0 |
| decode | 25 | 0 | 0 | 0 |
| execute | 31 | 0 | 0 | 0 |
| if | 9 | 0 | 0 | 0 |
| mc | 462 | 0 | 0 | 0 |
| memory | 60 | 0 | 0 | 0 |
| pipeline_logic | 406 | 0 | 0 | 0 |
| plic | 137 | 1 | 0 | 0 |
| prom | 2 | 0 | 0 | 2 |
| reg | 2290 | 273 | 0 | 0 |
| CSR | 1713 | 17 | 0 | 0 |
| GPR | 577 | 256 | 0 | 0 |
| rexmem | 141 | 0 | 0 | 0 |
| ridex | 786 | 32 | 0 | 0 |

**Table 5**
Implementation, new PMU.

| Component | Slice LUTs | F7 Muxes | F8 Muxes | Block RAM |
|---|---|---|---|---|
| RV32 | 7746 | 306 | 71 | 2 |
| clint | 20 | 0 | 0 | 0 |
| decode | 1 | 0 | 0 | 0 |
| execute | 31 | 0 | 0 | 0 |
| if | 9 | 0 | 0 | 0 |
| mc | 377 | 0 | 0 | 0 |
| memory | 106 | 0 | 0 | 0 |
| pipeline_logic | 354 | 14 | 7 | 0 |
| plic | 38 | 0 | 0 | 0 |
| prom | 0 | 0 | 0 | 2 |
| reg | 3735 | 256 | 64 | 0 |
| CSR | 3212 | 0 | 64 | 0 |
| GPR | 523 | 256 | 0 | 0 |
| rexmem | 141 | 0 | 0 | 0 |
| ridex | 786 | 32 | 0 | 0 |

masking. The new specification has added filters that indicate which traps can be enabled or disabled.

A power supply has been used to measure on-board consumption, and compare the old with the new PMU. First, a measurement of the base power consumption of the board was made, resulting in 0.168 A. Another measurement of the base power consumption with USB connection to a hardware server was done, the result in this case is 0.225 A.

Now the comparison is shown. For each PMU, the consumption has been measured with an external power supply, while running a program repeatedly, or interacting with the bootloader. For the old PMU, a value of 0.434 A has been recorded while interacting with the bootloader, and a value of 0.476 A has been reached while repeatedly executing a program in binary. On the other hand, for the case of the new PMU, interacting with the bootloader is 0.450 A, while repeatedly loading a program in binary is 0.494 A. Table 7 shows the amperage and power for each case described, using 5 V.

**Table 6**
Synthesis, new PMU.

| Component | Slice LUTs | F7 Muxes | F8 Muxes | Block RAM |
|---|---|---|---|---|
| RV32 | 8351 | 275 | 7 | 2 |
| clint | 76 | 0 | 0 | 0 |
| decode | 304 | 0 | 0 | 0 |
| execute | 634 | 1 | 0 | 0 |
| if | 38 | 0 | 0 | 0 |
| mc | 206 | 0 | 0 | 0 |
| memory | 125 | 0 | 0 | 0 |
| pipeline_logic | 409 | 0 | 0 | 0 |
| plic | 84 | 0 | 0 | 0 |
| prom | 2 | 0 | 0 | 2 |
| reg | 5516 | 274 | 7 | 0 |
| CSR | 4812 | 18 | 7 | 0 |
| GPR | 640 | 256 | 0 | 0 |
| rexmem | 112 | 0 | 0 | 0 |
| ridex | 234 | 0 | 0 | 0 |

**Table 7**
Comparison of the power consumption for both PMUs, in different operating modes.

| Test | Current (A) | Power (W) |
|---|---|---|
| Baseline | 0.168 | 0.840 |
| USB connection | 0.225 | 1.125 |
| Bootloader (Old PMU) | 0.434 | 2.170 |
| Programming (Old PMU) | 0.476 | 2.380 |
| Bootloader (New PMU) | 0.450 | 2.250 |
| Programming (New PMU) | 0.494 | 2.470 |

The power consumption is higher for the case of the new PMU, both when printing text on the screen and when loading a program.

One final consideration is the potential impact of the implemented modifications on the existing processor's performance. Notably, the critical path remained unaffected by these adjustments. This is because the combinational logic introduced is minimal compared to the length of the pipeline stages. For instance, examining each of the modifications, the additional logic for the changes to filter counter increments consists of only two levels of combinational logic, with its input and output routed through a register, thus rendering it negligible in comparison with the full logic of a stage. Similarly, the alterations to support overflow triggering employ logic analogous to that already in use for existing interrupts, as seen in Fig. 3, ensuring that its length never approached that of the critical path. Furthermore, timing analyses conducted before and after the implementation of the extension confirmed that the critical path remained unaltered.

## 5. Conclusions

In this paper, the update of the *RISC-V* privileged specification, which is implemented on the CPU, has been carried out. This specification refers mainly to the PMU, in particular to the supported instructions or registers.

A new interruption has been added, which is triggered when overflow is detected in any of the implemented counters. These counters are each associated to an event within the CPU.

In addition, the possibility of masking all interrupts for this reason at once (using the LCOFIE bit) has been added, as well as the possibility of enabling or disabling interrupts associated with specific counters (using the OF, MINH and UINH bits).

To check that the changes work correctly, a complete refactoring of the trap handler has been made in C code, unifying all supported traps, including this new interrupt. In addition, four tests were carried out in order to make sure that the overflow interruption triggers correctly, and the masking of interruptions on different privilege modes worked successfully.

Finally, the resources used in the new PMU were compared with those of the old PMU. The results show how resource and power utilization has increased in the updated PMU, as expected, given the increase in CSRs used and the additional logic required. With the addition of the changes a new functionality, a countdown timer, is added to the system.

## Funding

This work has been supported by "Programa Investigo" of the Autonomous Community of Madrid, funded by the European Union in the framework of the Recovery, Transformation and Resilience Plan [22].

## Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Andrea Fernandez Gallego reports financial support was provided by Comunidad de Madrid, Programa Investgo. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Data availability

Data will be made available on request.

## References

[1] RISC-V: a rising star in space, 2023, http://microelectronics.esa.int/papers/PresentationSummit-EUR-RISC-V-RisingStarInSpace-2023-06-08.pdf. (Accessed 8 September 2023).

[2] S. Di Mascio, A. Menicucci, G. Furano, C. Monteleone, M. Ottavi, The case for RISC-V in space, in: S. Saponara, A. De Gloria (Eds.), Applications in Electronics Pervading Industry, Environment and Society, Springer International Publishing, Cham, 2019, pp. 319–325.

[3] A. Waterman, Design of the RISC-V Instruction Set Architecture (Ph.D. thesis), EECS Department, University of California, Berkeley, 2016, URL http://www2.eecs.berkeley.edu/Pubs/TechRpts/2016/EECS-2016-1.html.

[4] M. Kovač, L. Dragić, B. Malnar, F. Minervini, O. Palomar, C. Rojas, M. Olivieri, J. Knezović, M. Kovač, FAUST: Design and implementation of a pipelined RISC-V vector floating-point unit, Microprocess. Microsyst. 97 (2023) 104762, http://dx.doi.org/10.1016/j.micpro.2023.104762, URL https://www.sciencedirect.com/science/article/pii/S014193312300008X.

[5] Space research group, 2023, https://www.uah.es/es/investigacion/unidades-de-investigacion/grupos-de-investigacion/Grupo-de-Investigacion-Espacial-Space-Research-Group./. (Accessed 10 February 2023).

[6] I. Gamino del Río, A. Martínez Hellín, Ó. R. Polo, M. Jiménez Arribas, P. Parra, A. da Silva, J. Sánchez, S. Sánchez, A RISC-V processor design for transparent tracing, Electronics 9 (11) (2020) 1873, http://dx.doi.org/10.3390/electronics9111873, URL https://www.mdpi.com/2079-9292/9/11/1873. (Visited 29 May 2023).

[7] N.-J. Wessman, F. Malatesta, J. Andersson, P. Gomez, M. Masmano, V. Nicolau, J.L. Rhun, G. Cabo, F. Bas, R. Lorenzo, O. Sala, D. Trilla, J. Abella, De-RISC: the first RISC-V space-grade platform for safety-critical systems, in: 2021 IEEE Space Computing Conference, SCC, 2021, pp. 17–26, http://dx.doi.org/10.1109/SCC49971.2021.00010.

[8] S. Sánchez, M. Prieto, Ó.R. Polo, P. Parra, A.d. Silva, Ó. Gutiérrez, R. Castillo, J. Fernández, J. Rodríguez-Pacheco, HW/SW co-design of the instrument control unit for the energetic particle detector on-board solar orbiter, Adv. Space Res. 52 (6) (2013) 989–1007, http://dx.doi.org/10.1016/j.asr.2013.05.029, URL https://www.sciencedirect.com/science/article/pii/S0273117713003360.

[9] J. Rodríguez-Pacheco, R.F. Wimmer-Schweingruber, G.M. Mason, G.C. Ho, S. Sánchez-Prieto, M. Prieto, C. Martín, H. Seifert, G.B. Andrews, S.R. Kulkarni, L. Panitzsch, S. Boden, S.I. Böttcher, I. Cernuda, R. Elftmann, F. Espinosa Lara, R. Gómez-Herrero, C. Terasa, J. Almena, S. Begley, E. Böhm, J.J. Blanco, W. Boogaerts, A. Carrasco, R. Castillo, A. Da Silva Fariña, V. De Manuel González, C. Drews, A.R. Dupont, S. Eldrum, C. Gordillo, O. Gutiérrez, D.K. Haggerty, J.R. Hayes, B. Heber, M.E. Hill, M. Jüngling, S. Kerem, V. Knierim, J. Köhler, S. Kolbe, A. Kulemzin, D. Lario, W.J. Lees, S. Liang, A. Martínez Hellín, D. Meziat, A. Montalvo, K.S. Nelson, P. Parra, R. Paspirgilis, A. Ravanbakhsh, M. Richards, O. Rodríguez-Polo, A. Russu, I. Sánchez, C.E. Schlemm, B. Schuster, L. Seimetz, J. Steinhagen, J. Tammen, K. Tyagi, T. Varela, M. Yedla, J. Yu, N. Agueda, A. Aran, T.S. Horbury, B. Klecker, K.-L. Klein, E. Kontar, S. Krucker, M. Maksimovic, O. Malandraki, C.J. Owen, D. Pacheco, B. Sanahuja, R. Vainio, J.J. Connell, S. Dalla, W. Dröge, O. Gevin, N. Gopalswamy, Y.Y. Kartavykh, K. Kudela, O. Limousin, P. Makela, G. Mann, H. Önel, A. Posner, J.M. Ryan, J. Soucek, S. Hofmeister, N. Vilmer, A.P. Walsh, L. Wang, M.E. Wiedenbeck, K. Wirth, Q. Zong, The energetic particle detector: Energetic particle instrument suite for the solar orbiter mission, Astron. Astrophys. 642 (2020) A7, http://dx.doi.org/10.1051/0004-6361/201935287, URL https://www.aanda.org/10.1051/0004-6361/201935287. (Visited 30 May 2023).

[10] The energetic particle detector, 2023, https://espada.uah.es/epd/index.php. (Accessed 10 February 2023).

[11] B. Sprunt, The basics of performance-monitoring hardware, IEEE Micro 22 (4) (2002) 64–71, http://dx.doi.org/10.1109/MM.2002.1028477.

[12] The RISC-V instruction set manual volume I: Unprivileged ISA, 2023, https://riscv.org/wp-content/uploads/2019/12/riscv-spec-20191213.pdf. (Accessed 10 May 2023).

[13] M.J. Arribas, A.M. Hellín, M.P. Mateo, I.G. del Río, A.F. Gallego, O.R. Polo, A. da Silva, P. Parra, S. Sánchez, Design and implementation of a synchronous hardware performance monitor for a RISC-V space-oriented processor, 2024, http://dx.doi.org/10.48550/arXiv.2404.05389, URL http://arxiv.org/abs/2404.05389, Preprint, arXiv:2404.05389 [cs] version: 1. (Visited 14 April 2024).

[14] J. Matijevic, E. Dewell, Anomaly recovery and the mars exploration rovers, 2006, http://dx.doi.org/10.2514/6.2006-5957.

[15] Y. Ando, S. Honda, H. Takada, M. Edahiro, Design space exploration of control system with hardware-implemented interrupt handler, in: 2015 6th International Conference of Information and Communication Technology for Embedded Systems, IC-ICTES, 2015, pp. 1–6, http://dx.doi.org/10.1109/ICTEmSys.2015.7110821.

[16] J. Biesiadecki, C. Leger, M. Maimone, Tradeoffs between directed and autonomous driving on the mars exploration rovers, Int. J. Robot. Res. 26 (2007) 91–104, http://dx.doi.org/10.1007/978-3-540-48113-3_23.

[17] RISC-V count overflow and mode-based filtering extension (sscofpmf), 2024, https://drive.google.com/file/d/1RiAIOVoN1E7bv6_kEzcgATkhbeUdqu5t/view. (Accessed 04 May 2024).

[18] S. Di Mascio, A. Menicucci, E. Gill, G. Furano, C. Monteleone, Leveraging the openness and modularity of RISC-V in space, J. Aerosp. Inf. Syst. 16 (11) (2019) 454–472, http://dx.doi.org/10.2514/1.I010735, arXiv:https://doi.org/10.2514/1.I010735.

[19] J. Dean, J. Hicks, C. Waldspurger, W. Weihl, G. Chrysos, ProfileMe: hardware support for instruction-level profiling on out-of-order processors, in: Proceedings of 30th Annual International Symposium on Microarchitecture, 1997, pp. 292–302, http://dx.doi.org/10.1109/MICRO.1997.645821.

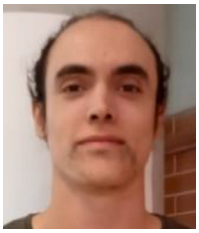
[20] T. Scheipel, F. Mauroner, M. Baunach, System-aware performance monitoring unit for RISC-v architectures, in: 2017 Euromicro Conference on Digital System Design, DSD, 2017, pp. 86–93, http://dx.doi.org/10.1109/DSD.2017.28.

[21] RISC-V cycle and instret privilege mode filtering (smcntrpmf), 2023, https://drive.google.com/file/d/1T33-7wnEo8Lar3n0k33ejW7_cROt0H0k/view. (Accessed 11 August 2023).

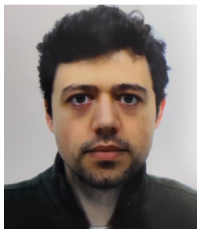
[22] Programa investigo, 2023, https://sede.comunidad.madrid/ayudas-becas-subvenciones/programa-investigo. (Accessed 24 August 2023).



**Andrea Fernández Gallego** is a recently graduated student in the Master's degree: Science and Technology from Space. At the same time, she was a member of the Space Research Group in Alcalá University, where she developed this paper. She currently works as FPGA design and verification engineer.



**Miguel Jiménez Arribas** obtained his M.Sc. degree in Space Science and Technology from the University of Alcalá (UAH), Spain, in 2019, and is currently pursuing his Ph.D. in Space Research and Astrobiology at that same university. Since 2019, he has been participating with the Department of Computer Engineering and the Space Research Group (SRG) within UAH, collaborating in different projects focused on the RISC-V architecture, researching new ways to improve its reliability for space systems. His research interests include computer architecture, on-board satellite hardware and software, and real-time embedded systems.



**Iván Gamino del Río** is a dedicated Ph.D. student in Space Research with over 5 years of expertise in VHDL FPGA Design, low-level programming, and system behavior analysis. Proficient in designing, developing, and testing customized RISC-V processing cores, memory, and bus interfaces. Possesses a keen interest in exploring diverse architectures, including SPARC (LEON family), RISC-V (NOEL-V), and ARM (Cortex family). Committed to ongoing knowledge enhancement to excel as a Computer Engineer and become an influential educator in the field.



**Agustín Martínez Hellín** received his M.S. degree from the Universidad Politécnica de Madrid (UPM) in 1986 and his Ph.D. degree from the Universidad de Alcalá in 2001. He is currently Professor of Computer Engineering at the University of Alcalá in Spain, where he conducts research and teaches in the areas of DSP, Computer Architecture, and Embedded Systems. He has held several technical and management positions at Telettra and Alcatel. He served as the Head of the Computer Engineering Department, Director of the "Master in Science and Technology from the Space" and director of Knowledge transfer at Alcalá University till now.



**Dr. Manuel Prieto Mateo** is currently associate professor in the University of Alcalá, Spain. He obtained his degree in Electronics Engineering in 1999 and his Ph.D. in 2005. His research interests are digital electronics development based on FPGAs and radiocommunications. He has leaded several national and international research projects in the field of space hardware and software development. He was the Project Manager of the Energetic Particle Detector (EPD) instrument on board ESA/NASA Solar Orbiter mission. Dr. Prieto is also involved in research projects in the Antarctica for the automation of measurement stations.



**Óscar R. Polo** received the M.S. degree in physics from the Universidad del Pais Vasco, Leioa, Spain, in 1994 and the Ph.D. degree in physics from the Universidad Complutense de Madrid, Madrid, Spain, in 2003. Since 2004, he has been with the Computing Engineering Department, University de Alcalá. He is currently an Assistant Professor of embedded and real-time systems. His research interests include computer architecture, satellite on-board software, model driven engineering, and embedded realtime systems. He has actively participated in several research projects in the area of computer engineering of satellite platforms in NANOSAT-01, NANOSAT-1B, Solar Orbiter, and Euclid missions.



**Antonio Da Silva** was born in Germany. He received the M.Sc. and Ph.D. degrees in computer engineering from Universidad de Alcalá, Alcalá de Henares, Spain, in 2001 and 2015, respectively. He has worked as an Embedded Software Developer for primary radar systems and since 1991 he lectures in the field of computing science. He has worked with the Space Research Group, Universidad de Alcalá on projects related to the development of the energetic particle detector (EPD) on-board Solar Orbiter. His research interests include serious games for education, fault tolerant systems design and critical software early development and verification.



**Pablo Parra** received his Ph.D. in Information and Communication Technologies from the University of Alcalá in 2012. Since 2006, he has been working with the Computer Engineering Department and the Space Research Group (SRG) of the University of Alcalá. His research interests include component-based software engineering and model-driven engineering applied to the field of real-time embedded systems. He has taken part in numerous research projects in the field of on-board satellite software development, such as the NANOSAT programme, Solar Orbiter and Euclid.



**Sebastián Sánchez** received the M.S. degree in telecommunications engineering from the Universidad Politécnica de Madrid, Madrid, Spain, in 1994 and the Ph.D. degree in telecommunications engineering from the Universidad de Alcalá, Alcalá de Henares, Spain, in 1998. Since 1990, he has been with the Computing Engineering Department, University de Alcalá. He is a Full Professor of operating systems and computer architecture. His research interests include space instrumentation, embedded real-time systems, and mobile robots. He has actively participated in several national and international research projects in the area of hardware and software on board satellites such as SOHO, PHOTON, FUEGO 2, NANOSAT, Exomars, MICROSAT, Solar Orbiter, and Euclid.